IET Software

Research Article

IET Journals
The Institution of Engineering and Technology

# Process metrics for software defect prediction in object-oriented programs

ISSN 1751-8806
Received on 8th December 2018
Revised 13th July 2019
Accepted on 19th November 2019
E-First on 26th February 2020
doi: 10.1049/iet-sen.2018.5439
www.ietdl.org

*Qiao Yu*[1] ✉, *Shujuan Jiang*[2,3], *Junyan Qian*[4,5], *Lili Bo*[6], *Li Jiang*[2,3], *Gongjie Zhang*[1]

[1]*School of Computer Science and Technology, Jiangsu Normal University, Xuzhou, People's Republic of China*
[2]*School of Computer Science and Technology, China University of Mining and Technology, Xuzhou, People's Republic of China*
[3]*Engineering Research Center of Mine Digitalization of Ministry of Education, Xuzhou, People's Republic of China*
[4]*Guangxi Key Laboratory of Multi-Source Information Mining & Security, Guangxi Normal University, Guilin, People's Republic of China*
[5]*Guangxi Key Laboratory of Trusted Software, Guilin University of Electronic Technology, Guilin, People's Republic of China*
[6]*School of Information Engineering, Yangzhou University, Yangzhou, People's Republic of China*
✉ *E-mail: yuqiao@jsnu.edu.cn*

**Abstract:** Software evolution is an important activity in the life cycle of a modern software system. In the process of software evolution, the repair of historical defects and the increasing demands may introduce new defects. Therefore, evolution-oriented defect prediction has attracted much attention of researchers in recent years. At present, some researchers have proposed the process metrics to describe the characteristics of software evolution. However, compared with the traditional software defect prediction methods, the research on evolution-oriented defect prediction is still inadequate. Based on the evolution data of object-oriented programs, this study presented two new process metrics from the defect rates of historical packages and the change degree of classes. To show the effectiveness of the proposed process metrics, the authors made comparisons with the code metrics and other process metrics. An empirical study was conducted on 33 versions of nine open-source projects. The results showed that adding the proposed process metrics could improve the performance of evolution-oriented defect prediction effectively.

## 1 Introduction

Software evolution is an important activity in the life cycle of a modern software system, and it is a dynamic and continuous process [1]. During the long running of a software system, the maintainers need to make changes due to repaired defects, increased demand, and changed environment. All these activities are known as software evolution. In the process of software evolution, the maintainers may introduce new defects along with the repair of historical defects, and the increase of demands may also introduce new defects. Therefore, software evolution is an important way to produce new defects, and it is also very important for evolution-oriented defect prediction.

At present, traditional software defect prediction methods are mainly based on the code metrics (CM), such as McCabe [2], Halstead [3], and object-oriented CK [4], which are also known as product metrics. These metrics can describe the static characteristics of software. In practice, evolution data may also be essential for software defect prediction. Therefore, researchers have proposed the process metrics (PM), such as code changes [5, 6], to improve the performance of software defect prediction.

Generally, the distribution of defects tends to meet the rule of 80:20, which indicates that the majority of software defects (about 80%) are distributed in a small proportion of software modules (about 20%) [7]. For programs written in the object-oriented language (e.g. Java), it indicates that 80% of defects may be distributed in 20% of packages or classes. During the evolution of a package, the historical defects may be fixed in the previous version, but new defects may be produced in the current version. Especially for a package with a large number of classes, we cannot repair all defects during one update, and we cannot guarantee that it does not produce new defects along with the repair. Therefore, the defect rate of a package in the previous version may be related to that in the current version. Besides, many researchers have proposed code changes [5, 6] for software defect prediction. Similarly, the class changes may also indicate the defect changes during the evolution process.

Based on the above analysis, this paper presented two new PM from the defect rates of historical packages and the change degree of classes for object-oriented programs. To show the validity of the proposed PM (PPM), we conducted experiments on 33 versions of nine open-source projects. The results showed that adding the PPM could improve the performance of defect prediction effectively. Moreover, the PPM performed better than the CM mentioned in [8] and the code churn metrics (CCM) defined in [9].

The main contributions of this paper are as follows:

- Two new process metrics are proposed for object-oriented programs, which are extracted from the defect rates of historical packages and the change degree of classes.
- An empirical study is conducted on 33 versions of nine open-source projects. The results show that the PPM perform better than the CM and the CCM.

The remainder of this paper is organised as follows. Section 2 summarises the related work on evolution-oriented defect prediction. Section 3 describes the details of our approach, including the PPM and the process of evolution-oriented defect prediction. Section 4 gives an empirical study to show the validity of the PPM. Section 5 summarises the threats to validity. Section 6 draws conclusions and discusses future work.

## 2 Related work

In recent years, software defect prediction has attracted much attention of researchers in software engineering [10–13]. Traditional software defect prediction methods are mainly based on the CM, which can only describe the static characteristics of software. However, software defects often change along with the evolution of software in practical applications. As a result, the prediction model built on the CM may not be ideal on the evolution versions [14]. Therefore, it is very important to extract the PM to improve the performance of evolution-oriented defect prediction.

At present, many researchers have proposed evolution metrics. For example, from the perspective of code changes, Nagappan and Ball [5] applied the code churn to predict the defect density, and the experimental results showed that the relative code churn performed better than the absolute code churn in predicting the defect density. Moser *et al.* [15] compared change metrics and CM on the Eclipse project, and the results indicated that the change metrics performed better than the CM. Hassan [6] proposed the complexity metrics based on code changes, and they conducted experiments on six large open-source projects to show the validity of change complexity metrics.

Kpodjedo *et al.* [16] proposed the design evolution metrics (DEM), including the numbers of added, deleted, and modified attributes, methods, and relations. The results showed that combining DEM with traditional metrics could improve the identification of defective classes, and DEM could predict more defects within a given size of code. Bhattacharya *et al.* [17] proposed the graph-based approach to capture the product metrics and PM. The results indicated that the graph metrics could be used to predict bug severity, maintenance effort, and defect-prone releases. Moreover, Stanić and Afzal [18] investigated different combinations of CM and PM, and the results indicated that the combination of PM and static CM tended to improve the prediction performance. Graves *et al.* [19] extracted the PM from software change history, which performed better than the product metrics. Rahman and Devanbu [20] conducted a large number of experiments to compare the performance of PM (such as code changes, developer/committer information etc.) and CM. The results indicated that PM performed better than CM.

In addition, Madeyski and Jureczko [7] investigated the performance of PM, such as the number of revisions, distinct committers, modified lines, and defects in the previous version. Wang and Wang [21] presented two types of evolution metrics, including version and code levels, and the experimental results showed the validity of these evolution metrics. They also indicated that the evolution data in the recent version performed better than that in all historical versions. Liu *et al.* [22] used the historical version sequence of metrics to predict defects in continuous software versions.

In view of the software change, the researchers have proposed the change-level defect prediction, which is known as just-in-time defect prediction [23, 24]. Unlike the traditional defect prediction methods, the change-level defect prediction methods regard a change as an instance for training and predict whether new software changes introduce changes or not. For example, Yuan *et al.* [25] presented a prediction approach based on the source code changes, which was designed in statement level. The experiments were conducted on six open-source projects, and the results indicated that the proposed source code changes approach performed better than that of the file and transaction changes approaches. Yang *et al.* [26] used deep learning to predict the defect-prone changes, and they used a deep belief network algorithm to build a set of expressive features from a set of initial change features. The experimental results showed that the proposed approach performed better than the approach proposed by Kamei *et al.* [27].

For cross-version defect prediction, Kastro and Bener [28] proposed a model to predict the number of defects in the new version with respect to the previous stable version. Xu *et al.* [29] proposed a two-phase framework that combined the hybrid active learning and kernel PCA to address the data differences between two versions. The experimental results indicated that the proposed framework outperformed the baseline methods. Yang and Wen [30] investigated the performance of ridge regression and lasso regression for cross-version defect prediction, and the results showed that both models performed better than the linear regression [31] and negative binomial regression [32] models.

In this paper, we aim to extract new PM based on the evolution data in recent version, rather than that in all historical versions. For object-oriented programs, the classes of two neighbouring versions may be different due to the changes in the evolution process. According to the changes of classes between two neighbouring versions, we divide the classes into two types: common classes and new classes. The common classes have evolution data, but the new classes do not have evolution data. Therefore, we attempt to extract the PM based on the common classes of two neighbouring versions.

## 3 Our approach

The framework of our approach is shown in Fig. 1. Based on the evolution versions of package (two neighbouring versions of package), we first use text matching approach to identify common classes where the package name and the class name of two neighbouring versions are exactly the same. Second, we extract PM according to the changes of these common classes. Finally, we add the extracted PM to build new datasets for training and test. As we extract new PM, we should build new datasets for training and test.

### 3.1 Proposed process metrics

Based on the evolution data of object-oriented programs, we presented two PM from the defect rates of historical packages and the change degree of classes. First of all, we give the basic definitions as follows.

*Definition 1: (Evolution period):* For two neighbouring versions $V_{t-1}$ and $V_t$ of a project, the change process from the previous version $V_{t-1}$ to the current version $V_t$ is called an evolution period.

The evolution period refers to the change process from $V_{t-1}$ to $V_t$, but not the time taken from $V_{t-1}$ to $V_t$. For example, there are three versions of a project as $V_1$, $V_2$, and $V_3$. The change process from $V_1$ to $V_2$ can be called as an evolution period, and the change process from $V_2$ to $V_3$ can also be called as an evolution period.

*Definition 2: (Defect rate of a package):* The percentage ratio of the number of defective classes to the number of all classes in a package. The calculation is shown in the following formula:

$$\text{Defect rate of a package} = \frac{\text{the number of defective classes}}{\text{the number of all classes}} \times 100\% \quad (1)$$

During an evolution period, the defects in $V_{t-1}$ may be fixed or may continue to exist in $V_t$. For a certain package in $V_{t-1}$ and $V_t$, the number of defective classes may be reduced or increased.

For a package with a large number of classes, when the defect rate of this package is very high, the maintainers cannot be able to repair all defects in an evolution period, even they may introduce new defects. In this case, the defect rate of a package in the previous version may reveal the defect rate of the same package in the current version, as well as the probability that one class may be a defective class in the current version. Based on this, we present the first process metric.

*3.1.1 pcdc – the probability that one class may be a defective class:* The defect rate of a package in the previous version is used to measure the defect rate of the same package in the current version and measure the probability that one class may be a

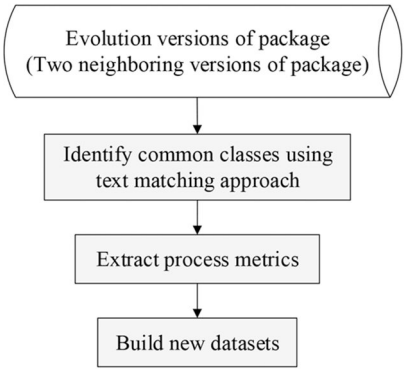


**Fig. 1** *Framework of our approach*

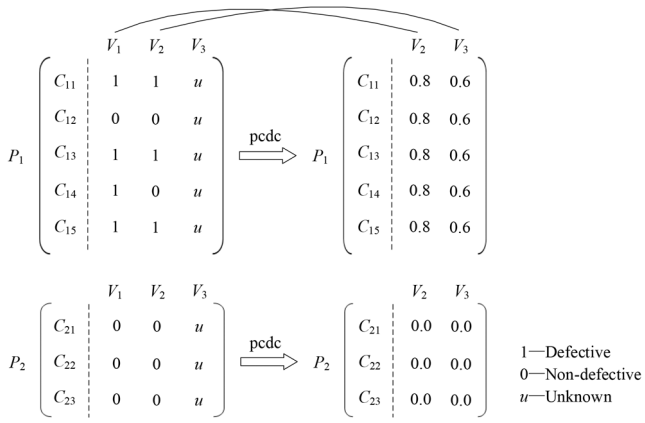


**Fig. 2** *Example to calculate pcdc*

| | $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ | $a_7$ | $a_8$ | $a_9$ | $a_{10}$ | pccm |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $V_1$ | 2 | 10 | 0 | 2.5 | 1 | 6 | 30 | 1.0 | 4 | 0 | |
| $V_2$ | 3 | 18 | 2 | 2.5 | 1 | 4 | 21 | 1.0 | 2 | 0 | 0.6 |
| $V_3$ | 2 | 12 | 1 | 5.0 | 0 | 9 | 50 | 1.0 | 2 | 1 | 0.8 |

**Fig. 3** *Example to calculate pccm*

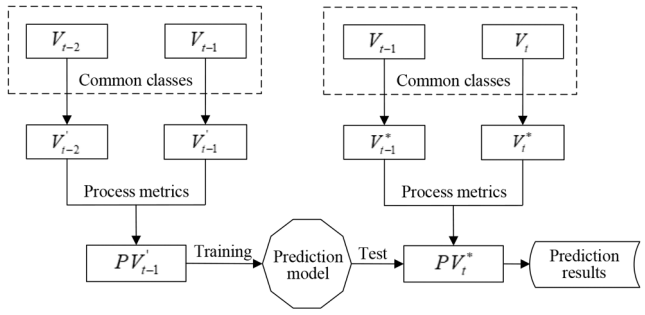


**Fig. 4** *Process of evolution-oriented defect prediction*

defective class in the current version. It should be noted that the proposed pcdc metric is designed in class granularity, but not in package granularity.

To better understand the calculation of pcdc, we take an example as illustrated in Fig. 2.

As displayed in Fig. 2, there are three versions ($V_1$, $V_2$, and $V_3$) and two packages ($P_1$ and $P_2$). $V_1$ and $V_2$ represent two historical versions, and $V_3$ represents the current version. There are five classes ($C_{11}$–$C_{15}$) in package $P_1$ and three classes ($C_{21}$–$C_{23}$) in package $P_2$. In particular, '0' represents a non-defective class, and '1' represents a defective class. Especially, '*u*' represents unknown, because we do not know the class label of the current version $V_3$. We can see that there are four defective classes and one non-defective class in package $P_1$ of $V_1$, so the defect rate (the ratio of defective classes to all classes) is $4/5 = 0.8$. So we mark the value of pcdc as 0.8 for $V_2$. Moreover, there is no defective class in package $P_2$ of $V_1$, $V_2$, and $V_3$, so we mark the value of pcdc as 0.0 for $V_2$ and $V_3$.

CM can describe the static characteristics of software, and each version can be represented by CM. When comparing the same classes of two neighbouring versions $V_{t-1}$ and $V_t$, we can get the changes of all CM. If a class changed on most of CM in an evolution period, it was considered that the change degree of this class was high. In other words, the evolution degree of this class was high. The higher evolution degree of a class indicated that the class might contain more defects. Based on this, we present the second process metric.

*3.1.2 pccm – the percentage of changed CM:* Comparing the previous version $V_{t-1}$ with the current version $V_t$, pccm represents the ratio of the number of changed CM to the number of all CM. This metric can reflect the change degree of a class in an evolution period.

Fig. 3 shows an example to calculate pccm. Suppose that there are ten CM ($a_1$, $a_2$, …, $a_{10}$) in one class, and there are three versions as $V_1$, $V_2$, and $V_3$. Comparing the values of these metrics between $V_1$ and $V_2$, there are six changed metrics ($a_1$, $a_2$, $a_3$, $a_6$, $a_7$, $a_9$). So, the value of pccm for $V_2$ is $6/10 = 0.6$. Similarly, comparing the values of metrics between $V_2$ and $V_3$, there are eight changed metrics ($a_1$, $a_2$, $a_3$, $a_4$, $a_5$, $a_6$, $a_7$, $a_{10}$). So, the value of pccm for $V_3$ is $8/10 = 0.8$.

Based on the PPM, we can build new datasets for evolution-oriented defect prediction. The details are described as follows.

### 3.2 Process of evolution-oriented defect prediction

For two neighbouring versions $V_{t-1}$ and $V_t$, their classes may be different due to the changes in the evolution process. Thus, we need to process $V_{t-1}$ and $V_t$ to get their common classes, then we can extract the PM.

The three continuous versions of a project were considered and symbolically were represented as $V_{t-2}$, $V_{t-1}$, and $V_t$ for showing the process of evolution-oriented defect prediction in Fig. 4. Furthermore, the three versions of a project contained the same CM.

The steps of evolution-oriented defect prediction are listed as follows:

Step 1: Process $V_{t-2}$ and $V_{t-1}$ to get their common classes using text matching approach, marked as $V'_{t-2}$ and $V'_{t-1}$.
Step 2: Extract the PPM (pcdc and pccm) based on $V'_{t-2}$ and $V'_{t-1}$.
Step 3: Add the PPM (pcdc and pccm) to $V'_{t-1}$, then get $PV'_{t-1}$.
Step 4: Perform the same process for $V_{t-1}$ and $V_t$, then get $PV^*_t$.
Step 5: $PV'_{t-1}$ is regarded as the training set to build the prediction model, and $PV^*_t$ is regarded as the test set.
Step 6: Get the prediction results.

It should be noted that the classes in $V'_{t-1}$ and $V^*_{t-1}$ may not be exactly the same.

## 4 Empirical study

To show the validity of the PPM, we conducted an empirical study on nine open-source projects from the Tera-PROMISE repository. All experiments were conducted on Open JDK 1.8 [33] and Weka 3.8 [34].

### 4.1 Experimental datasets

We used 33 versions of nine open-source projects for experiments, which were commonly used in defect prediction [7, 21, 35, 36]. These projects were designed at the class level, and each project contained at least three versions. In particular, the category label of the original datasets was the number of defects, so we converted the number of defects into binary classification. That is, the class without defect was considered to be non-defective, and the class with one or more defects was considered to be defective.

The details of these datasets are listed in Table 1 which shows the names of projects and their versions (columns 1-2). Then it shows the numbers of: (i) all samples, (ii) defective samples, and (iii) non-defective samples (columns 3-5). Finally, it shows the percentage defect rate (column 6).

For two neighbouring versions, their common classes have evolution data, but new classes do not have evolution data. In order to extract the PM, we need to process the datasets in Table 1 and get the common classes of two neighbouring versions. In our experiments, we used the text matching approach to identify the common classes between two neighbouring versions. That is to say, when the package name and the class name of two neighbouring versions were exactly the same, it was considered to be a common class. During the evolution of software, if one class changed its

**Table 1** Experimental datasets

| Project | Version | No. of all samples | No. of defective samples | No. of non-defective samples | Defect rate, % |
|---|---|---|---|---|---|
| Ant | 1.3 | 125 | 20 | 105 | 16.0 |
| | 1.4 | 178 | 40 | 138 | 22.5 |
| | 1.5 | 293 | 32 | 261 | 10.9 |
| | 1.6 | 351 | 92 | 259 | 26.2 |
| | 1.7 | 745 | 166 | 579 | 22.3 |
| Camel | 1.0 | 339 | 13 | 326 | 3.8 |
| | 1.2 | 608 | 216 | 392 | 35.5 |
| | 1.4 | 872 | 145 | 727 | 16.6 |
| | 1.6 | 965 | 188 | 777 | 19.5 |
| Jedit | 3.2 | 272 | 90 | 182 | 33.1 |
| | 4.0 | 306 | 75 | 231 | 24.5 |
| | 4.1 | 312 | 79 | 233 | 25.3 |
| | 4.2 | 367 | 48 | 319 | 13.1 |
| | 4.3 | 492 | 11 | 481 | 2.2 |
| Log4j | 1.0 | 135 | 34 | 101 | 25.2 |
| | 1.1 | 109 | 37 | 72 | 33.9 |
| | 1.2 | 205 | 189 | 16 | 92.2 |
| Lucene | 2.0 | 195 | 91 | 104 | 46.7 |
| | 2.2 | 247 | 144 | 103 | 58.3 |
| | 2.4 | 340 | 203 | 137 | 59.7 |
| Poi | 1.5 | 237 | 141 | 96 | 59.5 |
| | 2.0 | 314 | 37 | 277 | 11.8 |
| | 2.5 | 385 | 248 | 137 | 64.4 |
| | 3.0 | 442 | 281 | 161 | 63.6 |
| Synapse | 1.0 | 157 | 16 | 141 | 10.2 |
| | 1.1 | 222 | 60 | 162 | 27.0 |
| | 1.2 | 256 | 86 | 170 | 33.6 |
| Velocity | 1.4 | 196 | 147 | 49 | 75.0 |
| | 1.5 | 214 | 142 | 72 | 66.4 |
| | 1.6 | 229 | 78 | 151 | 34.1 |
| Xerces | 1.2 | 440 | 71 | 369 | 16.1 |
| | 1.3 | 453 | 69 | 384 | 15.2 |
| | 1.4 | 588 | 437 | 151 | 74.3 |

name or moves to another package due to software refactoring, it was regarded as a new class. In the datasets as shown in Table 1, there is no case that one class changes its name or moves to another package. Therefore, the text matching approach we used to identify the common classes will not affect the following experimental results.

The processed datasets are shown in Table 2. The samples in one processed dataset are the common classes with its previous version. So, the third column (No. of all samples) in Table 2 indicates the number of common classes. We take Ant as an example to show how to get the processed datasets. The example is displayed in Fig. 5.

As shown in Fig. 5, the number of samples in Ant-1.4* is the number of common classes of Ant-1.3 and Ant-1.4, and the number of samples in Ant-1.5* is the number of common classes of Ant-1.4 and Ant-1.5, and so on. For example, there are 125 common classes between the samples in Ant-1.3 and Ant-1.4 and 168 common classes between the samples in Ant-1.4 and Ant-1.5. Accordingly, we can get the processed datasets in Table 2 based on the datasets in Table 1.

Each dataset in Table 2 contains 20 features, which can be represented by CM in the class level. It indicates that each sample in these datasets represents a class in one project. The details of these CM are shown in Table 3. In addition, we also used 20 CCM as PM defined in [9], which represent the absolute change of CM (as listed in Table 3) between two neighbouring versions. The PPM are shown in Table 4.

### *4.2 Experimental design*

To comprehensively show the validity of the PPM, we designed the following three research questions (RQs).

*RQ1:* Based on the common classes of two neighbouring versions, whether there is a correlation between the defect rates of the same packages?

The pcdc metric is designed to use the defect rate of a package in the previous version to measure the defect rate of the same package in the current version, thereby measuring the probability that one class may be a defective class in the current version. In practice, whether there is a correlation between the defect rates of packages for two neighbouring versions? To answer this question, we use the Pearson's $r$ [37] to measure the correlation of the defect rate of a package between two neighbouring versions.

*RQ2:* Whether the PPM (pcdc and pccm) could improve the performance of defect prediction?

We conducted experiments on CM, CM + CCM, and CM + PPM. CM represents 20 CM in Table 3. CM + CCM represents 20 CM and 20 CCM. CM + PPM represents 20 CM and two PPM (pcdc and pccm). For two neighbouring and processed versions, we used the previous version as the training set, and the current version as the test set, then we compared the results of CM, CM + CCM, and CM + PPM. Moreover, we conducted feature selection on the training set by using the correlation-based feature selection (CFS) approach [38] and best-first search algorithm, then we also compared the results of CM, CM + CCM, and CM + PPM.

*RQ3:* Which is the best among the three combinations of CM + PPM, CM + pcdc, and CM + pccm?

**Table 2** Processed datasets

| Project | Version | No. of all samples | No. of defective samples | No. of non-defective samples | Defect rate, % |
|---|---|---|---|---|---|
| Ant | 1.4* | 125 | 22 | 103 | 17.6 |
| | 1.5* | 168 | 23 | 145 | 13.7 |
| | 1.6* | 292 | 72 | 220 | 24.7 |
| | 1.7* | 350 | 116 | 234 | 33.1 |
| Camel | 1.2* | 262 | 87 | 175 | 33.2 |
| | 1.4* | 569 | 117 | 452 | 20.6 |
| | 1.6* | 857 | 171 | 686 | 20.0 |
| Jedit | 4.0* | 265 | 62 | 203 | 23.4 |
| | 4.1* | 291 | 77 | 214 | 26.5 |
| | 4.2* | 291 | 41 | 250 | 14.1 |
| | 4.3* | 224 | 5 | 219 | 2.2 |
| Log4j | 1.1* | 98 | 33 | 65 | 33.7 |
| | 1.2* | 103 | 95 | 8 | 92.2 |
| Lucene | 2.2* | 192 | 112 | 80 | 58.3 |
| | 2.4* | 235 | 158 | 77 | 67.2 |
| Poi | 2.0* | 224 | 26 | 198 | 11.6 |
| | 2.5* | 314 | 219 | 95 | 69.7 |
| | 3.0* | 382 | 261 | 121 | 68.3 |
| Synapse | 1.1* | 152 | 45 | 107 | 29.6 |
| | 1.2* | 219 | 73 | 146 | 33.3 |
| Velocity | 1.5* | 155 | 84 | 71 | 54.2 |
| | 1.6* | 209 | 65 | 144 | 31.1 |
| Xerces | 1.3* | 433 | 62 | 371 | 14.3 |
| | 1.4* | 328 | 209 | 119 | 63.7 |

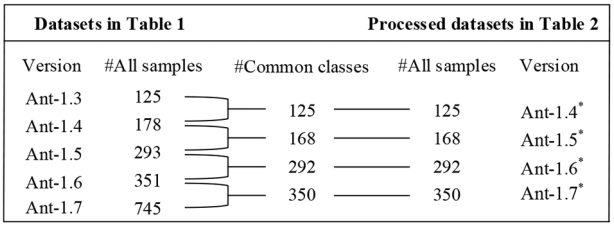


**Fig. 5** *Example to get the processed datasets*

**Table 3** CM and descriptions

| CM | Description | CM | Description |
|---|---|---|---|
| wmc | weighted methods per class | loc | lines of code |
| dit | depth of inheritance tree | dam | data access metric |
| noc | number of children | moa | measure of aggregation |
| cbo | coupling between object classes | mfa | measure of functional abstraction |
| rfc | response for a class | cam | cohesion among methods of class |
| lcom | lack of cohesion in methods | ic | inheritance coupling |
| ca | afferent couplings | cbm | coupling between methods |
| ce | efferent couplings | amc | average method complexity |
| npm | number of public methods | max_cc | maximum McCabe's cyclomatic complexity |
| lcom3 | lack of cohesion in methods, different from lcom | avg_cc | average McCabe's cyclomatic complexity |

**Table 4** PPM and descriptions

| PPM | Description |
|---|---|
| pcdc | the probability that one class may be a defective class |
| pccm | the percentage of changed CM |

PPM represents pcdc + pccm.

In order to explore the independent predictive ability of pcdc and pccm, we further make comparisons of CM + PPM, CM + pcdc, and CM + pccm.

In our experiments, we used *K*-nearest neighbours (KNN) [39], logistic regression (LR) [40], and naive Bayes (NB) [41] as the prediction models. The reason is that they do not have the built-in feature selection approach [42], and the performance of these models is more stable with the class imbalance problem [43]. These models are implemented in Weka. For the KNN model, the parameter '*K*' is set to '10', and the parameter '*distanceWeighting*' is set to '1/distance'. We use the default parameters of LR and NB models. Actually, there are many prediction models, and we only select KNN, LR, and NB for experiments, which are commonly used in empirical studies [42, 44, 45].

**Table 5** Correlation of the defect rate between two neighbouring versions

| Project | Version | No. of common packages | Pearson's *r* |
|---|---|---|---|
| Ant | 1.3 versus 1.4 | 8 | 0.251 |
| | 1.4 versus 1.5 | 10 | −0.047 |
| | 1.5 versus 1.6 | 21 | 0.012 |
| | 1.6 versus 1.7 | 24 | 0.151 |
| Camel | 1.0 versus 1.2 | 36 | **0.432** |
| | 1.2 versus 1.4 | 70 | **0.743** |
| | 1.4 versus 1.6 | 108 | **0.493** |
| Jedit | 3.2 versus 4.0 | 15 | **0.746** |
| | 4.0 versus 4.1 | 17 | **0.735** |
| | 4.1 versus 4.2 | 18 | **0.663** |
| | 4.2 versus 4.3 | 17 | 0.155 |
| Log4j | 1.0 versus 1.1 | 11 | **0.928** |
| | 1.1 versus 1.2 | 10 | −0.012 |
| Lucene | 2.0 versus 2.2 | 10 | **0.608** |
| | 2.2 versus 2.4 | 12 | **0.443** |
| poi | 1.5 versus 2.0 | 17 | **0.496** |
| | 2.0 versus 2.5 | 19 | 0.303 |
| | 2.5 versus 3.0 | 20 | 0.231 |
| Synapse | 1.0 versus 1.1 | 22 | **0.669** |
| | 1.1 versus 1.2 | 30 | 0.144 |
| Velocity | 1.4 versus 1.5 | 22 | −0.089 |
| | 1.5 versus 1.6 | 24 | **0.501** |
| Xerces | 1.2 versus 1.3 | 24 | 0.227 |
| | 1.3 versus 1.4 | 14 | **0.439** |

For each project with several versions, the former version is regarded as the training set, and the next version is regarded as the test set. We use AUC [46] as the performance metric, which represents the area under the receiver operating characteristic curve. The value of AUC is between 0 and 1. The larger AUC indicates that the performance of one model is better. Jiang *et al.* [47] proved that AUC was more accurate and reliable than other metrics. Moreover, AUC has been widely used in empirical studies of software defect prediction [48–54].

### *4.3 Experimental results and analysis*

*4.3.1 Correlation of the defect rate between two neighbouring versions:* The pcdc metric has been proposed to measure the defect rate of the package in the current version using the defect rate of a package in the previous version. Here, the defect rate has measured the probability that one class might be a defective class in the current version. In other words, this metric indicates that the defect rates of the same packages may be similar between two neighbouring versions.

In order to show its rationality, we used the Pearson's $r$ to measure the correlation of the defect rate of a package between two neighbouring versions. The value of $r$ is between −1 and 1. The larger $|r|$ indicates that the correlation between two neighbouring versions is higher. Table 5 shows the number of common packages and Pearson's $r$ between two neighbouring versions.

We analysed the common packages of two neighbouring versions. Then, we calculated the defect rate of each package and calculated the Pearson's $r$ between two neighbouring versions.

As shown in Table 5, we used 'bold' to mark the values of $r$ with a medium or large correlation ($r > 0.400$). We find that more than half comparisons make a medium or large correlation. However, the value of $r$ may be lower for some project, especially for Ant. Moreover, the value of $r$ may be affected by the fluctuation of defect rates. For example, the defect rates of the processed versions in Jedit are 23.4, 26.5, 14.1, and 2.2%. The values of $r$ are 0.746, 0.735, 0.663, and 0.155.

On the whole, we concluded that there was a certain correlation between the defect rates of the same packages for neighbouring versions, especially for those stable evolved projects. It indicated that the proposed pcdc metric could be able to indicate the probability of defects. However, when the defect rates of the same packages between two neighbouring versions were very different, the effectiveness of pcdc metric may be affected to a certain extent. This may be related to the evolution process of projects.

*4.3.2 Comparisons of CM, CM + CCM, and CM + PPM:* In order to show the validity of PPM for defect prediction, we selected the datasets in Table 2 as experimental subjects and designed experiments with CM, CM + CCM, and CM + PPM for comparisons.

As listed in Table 6, 'Training set→Test set' represents the prediction process from the training set to the test set. For example, 'Ant-1.4*→Ant-1.5*' represents that Ant-1.4 is regarded as the training set, and Ant-1.5 is regarded as the test set. 'Bold' represents the max of CM, CM + CCM, and CM + PPM. Moreover, we conducted feature selection on the training set by using the CFS approach and best-first search algorithm, then we made comparisons of CM, CM + CCM, and CM + PPM. The results are listed in Table 7. Similarly, 'bold' represents the max of CM, CM + CCM, and CM + PPM.

As shown in Tables 6 and 7, we find that CM + PPM performs better than CM + CCM on most predictions, but performs better than CM only on part of predictions. In addition, CM performs better than CM + CCM on most predictions.

To further show the effectiveness of the PPM for defect prediction, we applied the Wilcoxon signed-rank test [55], a non-parametric test for two related variables, to determine the statistical significance of two approaches [56]. The significance level $\alpha = 0.05$. Moreover, we applied the Cohen's $d$ [57] to measure the effect size of differences between two approaches. Cohen presented three levels of effect size as $d = 0.2$ (small, S), $d = 0.5$ (medium, M), and $d = 0.8$ (large, L). All test results are listed in Tables 8 and 9.

According to the Wilcoxon signed-rank test results in Tables 8 and 9, CM + PPM performs better than CM on KNN and NB models. However, there may be no significant difference between CM + PPM and CM on LR model. CM + PPM outperforms CM + CCM on three models. Unfortunately, there may be no significant difference between CM + CCM and CM. From the results of Cohen's $d$, CM + PPM performs better than CM and CM + CCM

**Table 6** Comparisons of CM, CM + CCM, and CM + PPM (AUC)

| Prediction number | Training set→Test set | KNN model | | | LR model | | | NB model | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | CM | CM + CCM | CM + PPM | CM | CM + CCM | CM + PPM | CM | CM + CCM | CM + PPM |
| 1 | Ant-1.4*→Ant-1.5* | 0.813 | **0.841** | 0.804 | **0.713** | 0.610 | 0.697 | 0.780 | 0.643 | **0.798** |
| 2 | Ant-1.5*→Ant-1.6* | **0.729** | 0.703 | 0.702 | **0.672** | 0.623 | 0.665 | 0.759 | 0.747 | **0.765** |
| 3 | Ant-1.6*→Ant-1.7* | 0.849 | **0.854** | 0.853 | **0.820** | 0.705 | 0.811 | 0.817 | 0.797 | **0.819** |
| 4 | Camel-1.2*→Camel-1.4* | 0.658 | 0.622 | **0.774** | 0.663 | 0.554 | **0.849** | 0.619 | 0.595 | **0.821** |
| 5 | Camel-1.4*→Camel-1.6* | 0.692 | 0.667 | **0.793** | 0.642 | 0.592 | **0.759** | 0.641 | 0.639 | **0.720** |
| 6 | Jedit-4.0*→Jedit-4.1* | 0.837 | 0.842 | **0.869** | 0.819 | 0.629 | **0.839** | 0.807 | 0.799 | **0.860** |
| 7 | Jedit-4.1*→Jedit-4.2* | 0.822 | 0.836 | **0.860** | 0.871 | 0.842 | **0.881** | 0.840 | 0.799 | **0.854** |
| 8 | Jedit-4.2*→Jedit-4.3* | **0.900** | 0.883 | 0.887 | **0.903** | 0.865 | 0.889 | **0.887** | 0.880 | 0.880 |
| 9 | Log4j-1.1*→Log4j-1.2* | 0.497 | 0.533 | **0.567** | **0.655** | 0.626 | 0.609 | **0.601** | 0.597 | 0.586 |
| 10 | Lucene-2.2*→Lucene-2.4* | 0.583 | **0.668** | 0.625 | 0.572 | 0.477 | **0.609** | 0.705 | 0.660 | **0.723** |
| 11 | Poi-2.0*→Poi-2.5* | 0.403 | 0.494 | **0.753** | 0.332 | **0.418** | 0.261 | **0.474** | 0.470 | 0.471 |
| 12 | Poi-2.5*→Poi-3.0* | 0.737 | 0.732 | **0.775** | **0.781** | 0.706 | 0.774 | **0.802** | 0.756 | 0.791 |
| 13 | Synapse-1.1*→Synapse-1.2* | 0.677 | 0.684 | **0.695** | 0.545 | **0.595** | 0.525 | 0.634 | 0.645 | **0.657** |
| 14 | Velocity-1.5*→Velocity-1.6* | **0.704** | 0.662 | 0.682 | 0.735 | 0.674 | **0.752** | **0.721** | 0.715 | 0.711 |
| 15 | Xerces-1.3*→Xerces-1.4* | 0.568 | **0.579** | 0.568 | 0.492 | 0.431 | **0.587** | **0.755** | 0.753 | 0.752 |

**Table 7** Comparisons of CM, CM + CCM, and CM + PPM using feature selection (AUC)

| Prediction number | Training set→Test set | KNN model | | | LR model | | | NB model | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | CM | CM + CCM | CM + PPM | CM | CM + CCM | CM + PPM | CM | CM + CCM | CM + PPM |
| 1 | Ant-1.4*→Ant-1.5* | 0.758 | 0.756 | **0.805** | 0.801 | 0.733 | **0.810** | 0.794 | 0.677 | **0.809** |
| 2 | Ant-1.5*→Ant-1.6* | **0.736** | 0.725 | **0.736** | **0.813** | 0.800 | **0.813** | **0.821** | 0.812 | **0.821** |
| 3 | Ant-1.6*→Ant-1.7* | 0.819 | 0.821 | **0.846** | 0.846 | 0.846 | **0.847** | **0.830** | 0.810 | 0.825 |
| 4 | Camel-1.2*→Camel-1.4* | 0.593 | 0.683 | **0.858** | 0.594 | 0.647 | **0.853** | 0.573 | 0.656 | **0.812** |
| 5 | Camel-1.4*→Camel-1.6* | 0.719 | 0.663 | **0.811** | 0.669 | 0.633 | **0.770** | 0.626 | 0.609 | **0.786** |
| 6 | Jedit-4.0*→Jedit-4.1* | 0.799 | 0.757 | **0.841** | 0.803 | 0.771 | **0.864** | 0.795 | 0.799 | **0.872** |
| 7 | Jedit-4.1*→Jedit-4.2* | 0.822 | 0.821 | **0.861** | 0.859 | 0.840 | **0.881** | 0.837 | 0.810 | **0.864** |
| 8 | Jedit-4.2*→Jedit-4.3* | **0.869** | 0.856 | 0.837 | **0.907** | 0.857 | 0.878 | **0.897** | 0.871 | 0.891 |
| 9 | Log4j-1.1*→Log4j-1.2* | 0.449 | 0.521 | **0.597** | **0.561** | 0.537 | 0.480 | 0.589 | **0.603** | 0.555 |
| 10 | Lucene-2.2*→Lucene-2.4* | 0.529 | 0.545 | **0.592** | 0.685 | 0.679 | **0.704** | 0.596 | 0.606 | **0.662** |
| 11 | Poi-2.0*→Poi-2.5* | 0.490 | 0.663 | **0.719** | **0.322** | 0.321 | 0.290 | **0.398** | 0.395 | 0.365 |
| 12 | Poi-2.5*→Poi-3.0* | 0.735 | 0.735 | **0.785** | **0.787** | **0.787** | 0.777 | **0.802** | **0.802** | 0.795 |
| 13 | Synapse-1.1*→Synapse-1.2* | **0.698** | 0.696 | 0.652 | 0.630 | **0.660** | 0.658 | 0.707 | **0.710** | 0.649 |
| 14 | Velocity-1.5*→Velocity-1.6* | 0.738 | 0.705 | **0.739** | 0.718 | 0.715 | **0.734** | **0.732** | 0.722 | 0.722 |
| 15 | Xerces-1.3*→Xerces-1.4* | 0.587 | 0.593 | **0.620** | 0.714 | 0.718 | **0.732** | 0.690 | 0.690 | **0.693** |

**Table 8** Test results of CM, CM + CCM, and CM + PPM

| Prediction model | CM + PPM versus CM | | CM + CCM versus CM | | CM + PPM versus CM + CCM | |
|---|---|---|---|---|---|---|
| | Wilcoxon | Cohen's *d* | Wilcoxon | Cohen's *d* | Wilcoxon | Cohen's *d* |
| KNN | 0.022 | 0.398 (S) | 0.589 | 0.066 | 0.078 | 0.355 (S) |
| LR | 0.513 | 0.122 | 0.011 | −0.410 (S) | 0.020 | 0.521 (M) |
| NB | 0.112 | 0.221 (S) | 0.003 | −0.214 (S) | 0.006 | 0.440 (M) |

**Table 9** Test results of CM, CM + CCM, and CM + PPM using feature selection

| Prediction model | CM + PPM versus CM | | CM + CCM versus CM | | CM + PPM versus CM + CCM | |
|---|---|---|---|---|---|---|
| | Wilcoxon | Cohen's *d* | Wilcoxon | Cohen's *d* | Wilcoxon | Cohen' *d* |
| KNN | 0.008 | 0.558 (M) | 0.875 | 0.116 | 0.004 | 0.520 (M) |
| LR | 0.245 | 0.164 | 0.116 | −0.077 | 0.047 | 0.241 (S) |
| NB | 0.470 | 0.211 (S) | 0.249 | −0.060 | 0.084 | 0.278 (S) |

with middle or small effects, but CM + CCM may be comparative or inferior to CM on some predictions.

We concluded that adding the PPM into CM could improve the performance of defect prediction to a certain extent. It also indicated that the PPM were effective for defect prediction.

Based on the above results, we find that the effectiveness of PM may be related to the evolution pattern of a project. For example, as shown in Tables 6 and 7, CM + PPM performs better than CM significantly in Camel and Lucene projects. The reason is that there is a little change of defect rates between their neighbouring versions. In addition, CM + PPM outperforms CM on predictions of Jedit-4.0*→Jedit-4.1* and Jedit-4.1*→Jedit-4.2*, but CM outperforms CM + PPM on the prediction of Jedit-4.2*→Jedit-4.3*. The reason is that the defect rates of Jedit-4.0*, Jedit-4.1*, Jedit-4.2*, and Jedit-4.3* are 23.4, 26.5, 14.1, and 2.2%. We can see that the defect rate of Jedit-4.3* is considerably lower than other versions, which may affect the effectiveness of pcdc metric. Therefore, the defect rates of different versions may affect the performance of the PPM. Especially in the

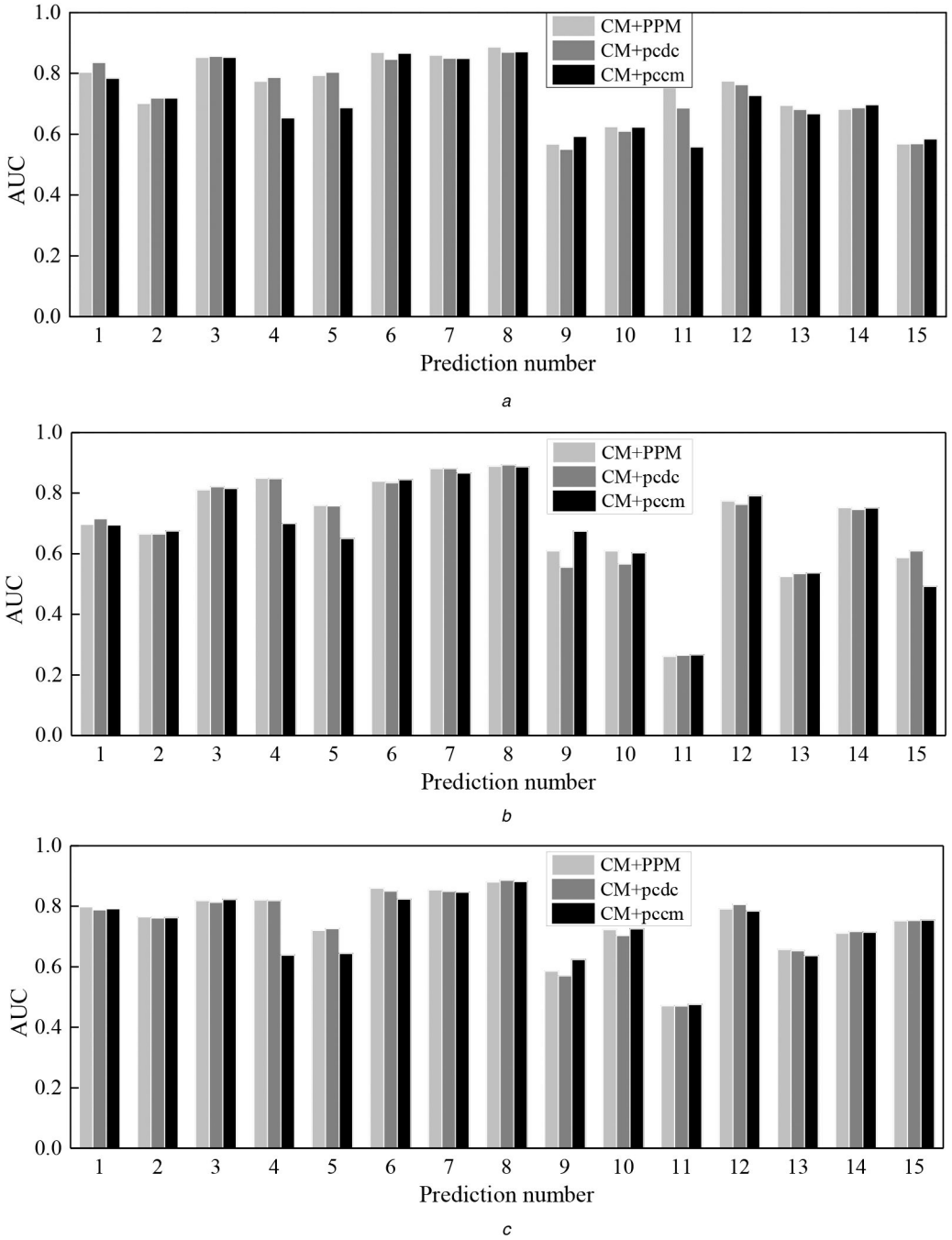


**Fig. 6** *Comparisons of CM + PPM, CM + pcdc, and CM + pccm*
***(a)*** KNN model, ***(b)*** LR model, ***(c)*** NB model

case of major changes in defect rates, the effectiveness of the PPM may be reduced.

Besides, we also find that CM + PPM performs better than CM on most predictions on KNN model, and only on part of predictions on LR and NB models, which indicate the differences on different prediction models. For example, on predictions of Poi-2.0*→Poi-2.5* and Poi-2.5*→Poi-3.0*, CM + PPM outperforms CM on KNN model, but inferior to CM on LR and NB models.

We concluded that the PPM could improve the performance of defect prediction effectively, especially for those stable evolved projects. In practice, we can select a certain percentage of samples from the current version as the test set to verify the effectiveness of PM. If the results show that the PPM are helpful to improve its performance, then we continue to predict other classes in the current version.

*4.3.3 Comparisons of CM + PPM, CM + pcdc, and CM + pccm:* The above experiments showed the combined performance of PPM (pcdc + pccm). In order to explore the independent predictive ability of pcdc and pccm, we further made comparisons of CM + PPM, CM + pcdc, and CM + pccm. We also conducted experiments on KNN, LR, and NB models. We used the bar charts to show the AUC of CM + PPM, CM + pcdc, and CM + pccm. As displayed in Fig. 6, the *x*-axis represents the prediction numbers of 1–15 as listed in Table 6, and the *y*-axis represents the value of AUC.

As can be seen from Fig. 6, CM + PPM is equivalent to or slightly better than CM + pcdc on most predictions, and CM + PPM is better than or equivalent to CM + pccm on most predictions. CM + pcdc is significantly better than CM + pccm on part of predictions. However, CM + pccm is significantly better than CM + pcdc on some predictions.

Based on the above experiments, we can conclude the following points, which can also answer the presented RQs.

*The answer to RQ1:* There is a certain correlation between the defect rates of packages for two neighbouring versions, especially for those stable evolved projects.

*The answer to RQ2:* When the defect rates of packages are stable between two neighbouring versions, the PPM can improve the performance of defect prediction effectively.

*The answer to RQ3:* CM + PPM is equivalent to or slightly better than CM + pcdc on most predictions, and CM + PPM is better than or equivalent to CM + pccm on most predictions.

In practice, for a project with multiple versions, we can select a part of classes (the common classes with the previous version) from the current version and label them by manual. Then, these classes are regarded as the validation set to verify the effectiveness of PM for defect prediction. If the results show that PM are helpful to improve its performance, we continue to predict other classes in the current version. On the contrary, if the validity of PM is not obvious, we can only use CM for defect prediction, so as to avoid the cost of extracting PM.

As a matter of fact, whether it is a code metric or a process metric, a single metric may be one-sided. There is no one metric that can get the best performance on any datasets, and reasonable combination of multiple metrics may achieve better performance.

## 5 Threats to validity

However, we find several threats to the validity of our experiments, which can be summarised into three aspects as follows.

- *Construct validity:* The number of common classes between two neighbouring versions may be the main threat to construct validity. The PPM focused on the common classes between two neighbouring versions. If the number of common classes is small, the proposed metrics may not work well, especially for highly evolved software systems.

  In addition, the previous study indicated that the evolution data of the previous version performed better than that of all historical versions [7]. Therefore, we extracted PM based on the evolution data in the previous version, rather than that in all historical versions. We conducted a large number of experiments to show the effectiveness of the PPM for defect prediction. In practice, each version may provide some useful data for defect prediction. How to extract valuable PM from one or more historical versions is worthy of study.
- *Internal validity:* The prediction models we used may be the threat to internal validity. We used KNN, LR, and NB for full experiments, and the experimental results indicated the differences between different prediction models. So, the performance of the PPM should be explored on more models.
- *External validity:* The quality of datasets may be the threat to external validity. In our experiments, we selected nine open-source projects as the experimental subjects. They were all developed in Java and commonly used in software defect prediction. In particular, in order to extract PM and verify their validity, the selected projects contain at least three versions. The experimental results also indicated that the defect rates of datasets may affect the effectiveness of the PPM. Therefore, more datasets should be evaluated in the following work.

## 6 Conclusion and future work

Based on the evolution data of object-oriented programs, this paper presented two PM from the defect rates of historical packages and the change degree of classes. An empirical study was conducted on 33 versions of nine open-source projects. The results showed that adding the PPM could improve the performance of defect prediction effectively, especially when the defect rates were stable between neighbouring versions.

In particular, we used the common classes of neighbouring versions for experiments. The prediction of new classes will be explored in the following research. We will also attempt to extract more generalised PM for defect prediction.

## 7 Acknowledgments

This work was partly supported by the National Natural Science Foundation of China (Nos. 61902161, 61673384, and 61562015), the Natural Science Foundation of the Jiangsu Higher Education Institutions of China (No. 18KJB520016), the Guangxi Key Laboratory of Trusted Software (No. kx201704), and the Research Support Program for Doctorate Teachers of Jiangsu Normal University (Nos. 17XLR001 and 17XLR042).

## 8 References

[1] Kemerer, C.F., Slaughter, S.: 'An empirical approach to studying software evolution', *IEEE Trans. Softw. Eng.*, 1999, **25**, (4), pp. 493–509
[2] McCabe, T.J.: 'A complexity measure', *IEEE Trans. Softw. Eng.*, 1976, **SE-2**, (4), pp. 308–320
[3] Halstead, M.H.: '*Elements of software science*' (Elsevier, New York, USA, 1977)
[4] Chidamber, S.R., Kemerer, C.F.: 'A metrics suite for object oriented design', *IEEE Trans. Softw. Eng.*, 1994, **20**, (6), pp. 476–493
[5] Nagappan, N., Ball, T.: 'Use of relative code churn measures to predict system defect density'. Proc. Int. Conf. Software Engineering, Louis, Missouri, USA, May 2005, pp. 284–292
[6] Hassan, A.E.: 'Predicting faults using the complexity of code changes'. Proc. Int. Conf. Software Engineering, Vancouver, Canada, May 2009, pp. 78–88
[7] Madeyski, L., Jureczko, M.: 'Which process metrics can significantly improve defect prediction models? An empirical study', *Softw. Qual. J.*, 2015, **23**, (3), pp. 393–422
[8] Jureczko, M., Madeyski, L.: 'Towards identifying software project clusters with regard to defect prediction'. Proc. Int. Conf. Predictive Models in Software Engineering, Timisoara, Romania, September 2010
[9] Hall, G.A., Munson, J.C.: 'Software evolution: code delta and code churn', *J. Syst. Softw.*, 2000, **54**, pp. 111–118
[10] Catal, C.: 'Software fault prediction: A literature review and current trends', *Expert Syst. Appl.*, 2011, **38**, (4), pp. 4626–4636
[11] Hall, T., Beecham, S., Bowes, D., *et al.*: 'A systematic literature review on fault prediction performance in software engineering', *IEEE Trans. Softw. Eng.*, 2012, **38**, (6), pp. 1276–1304
[12] Malhotra, R.: 'A systematic review of machine learning techniques for software fault prediction', *Appl. Soft Comput.*, 2015, **27**, pp. 504–518
[13] Li, Z., Jing, X.Y., Zhu, X.: 'Progress on approaches to software defect prediction', *IET Softw.*, 2018, **12**, (3), pp. 161–175
[14] Shatnawi, R., Li, W.: 'The effectiveness of software metrics in identifying error-prone classes in post-release software evolution process', *J. Syst. Softw.*, 2008, **81**, (11), pp. 1868–1882
[15] Moser, R., Pedrycz, W., Succi, G.: 'A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction'. Proc. Int. Conf. Software Engineering, Leipzig, Germany, May 2008, pp. 181–190
[16] Kpodjedo, S., Ricca, F., Galinier, P., *et al.*: 'Design evolution metrics for defect prediction in object oriented systems', *Empir. Softw. Eng.*, 2011, **16**, (1), pp. 141–175
[17] Bhattacharya, P., Iliofotou, M., Neamtiu, I., *et al.*: 'Graph-based analysis and prediction for software evolution'. Proc. Int. Conf. Software Engineering, Zurich, Switzerland, June 2012, pp. 419–429
[18] Stanić, B., Afzal, W.: 'Process metrics are not bad predictors of fault proneness'. Proc. Int. Conf. Software Quality, Reliability and Security Companion, Prague, Czech Republic, July 2017, pp. 493–499
[19] Graves, T.L., Karr, A.F., Marron, J.S., *et al.*: 'Predicting fault incidence using software change history', *IEEE Trans. Softw. Eng.*, 2000, **26**, (7), pp. 653–661
[20] Rahman, F., Devanbu, P.: 'How, and why, process metrics are better'. Proc. Int. Conf. Software Engineering, San Francisco, CA, USA, May 2013, pp. 432–441
[21] Wang, D., Wang, Q.: 'Improving the performance of defect prediction based on evolution data', *J. Softw.*, 2016, **27**, (12), pp. 3014–3029
[22] Liu, Y., Li, Y., Guo, J., *et al.*: 'Connecting software metrics across versions to predict defects'. Proc. Int. Conf. Software Analysis, Evolution and Reengineering Campobasso, Italy, March 2018, pp. 232–243
[23] Yang, X., Lo, D., Xia, X., *et al.*: 'TLEL: a two-layer ensemble learning approach for just-in-time defect prediction', *Inf. Softw. Technol.*, 2017, **87**, pp. 206–220
[24] Pascarella, L., Palomba, F., Bacchelli, A.: 'Fine-grained just-in-time defect prediction', *J. Syst. Softw.*, 2019, **150**, pp. 22–36
[25] Yuan, Z., Yu, L., Liu, C.: 'Bug prediction method for fine-grained source code changes', *J. Softw.*, 2014, **25**, (11), pp. 2499–2517
[26] Yang, X., Lo, D., Xia, X., *et al.*: 'Deep learning for just-in-time defect prediction'. Proc. Int. Conf. Software Quality, Reliability and Security, Vancouver, BC, Canada, August 2015, pp. 17–26
[27] Kamei, Y., Shihab, E., Adams, B., *et al.*: 'A large-scale empirical study of just-in-time quality assurance', *IEEE Trans. Softw. Eng.*, 2013, **39**, (6), pp. 757–773
[28] Kastro, Y., Bener, A.B.: 'A defect prediction method for software versioning', *Softw. Qual. J.*, 2008, **16**, (4), pp. 543–562
[29] Xu, Z., Liu, J., Luo, X., *et al.*: 'Cross-version defect prediction via hybrid active learning with kernel principal component analysis'. Proc. Int. Conf. Software Analysis, Evolution and Reengineering, Campobasso, Italy, March 2018, pp. 209–220
[30] Yang, X., Wen, W.: 'Ridge and lasso regression models for cross-version defect prediction', *IEEE Trans. Reliab.*, 2018, **67**, (3), pp. 885–896
[31] Montgomery, D.C., Peck, E.A.: '*Introduction to linear regression analysis*' (Wiley, New York, 1982)
[32] McCullagh, P., Nelder, J.A.: '*Generalized linear models*' (Chapman and Hall, London, UK, 1989, 2nd edn.)
[33] 'Open JDK'. Available at http://openjdk.java.net/
[34] 'Weka'. Available at https://www.cs.waikato.ac.nz/ml/weka/

[35] Xia, X., Lo, D., Pan, S.J., *et al.*: 'HYDRA: massively compositional model for cross-project defect prediction', *IEEE Trans. Softw. Eng.*, 2016, **42**, (10), pp. 977–998
[36] Gupta, S., Gupta, A.: 'A set of measures designed to identify overlapped instances in software defect prediction', *Computing*, 2017, **99**, (9), pp. 889–914
[37] 'Pearson's r'. Available at https://en.wikipedia.org/wiki/Pearson_correlation_coefficient
[38] Hall, M.A.: 'Correlation-based feature selection for machine learning'. PhD thesis, The University of Waikato, 1999
[39] Aha, D.W., Kibler, D., Albert, M.K.: 'Instance-based learning algorithms', *Mach. Learn.*, 1991, **6**, (1), pp. 37–66
[40] Le Cessie, S., Van Houwelingen, J.C.: 'Ridge estimators in logistic regression', *Appl. Stat.*, 1992, **41**, (1), pp. 191–201
[41] John, G.H., Langley, P.: 'Estimating continuous distributions in Bayesian classifiers'. Proc. Annual Conf. Uncertainty in Artificial Intelligence, Montreal, Quebec, Canada, August 1995, pp. 338–345
[42] Gao, K., Khoshgoftaar, T.M., Wang, H., *et al.*: 'Choosing software metrics for defect prediction: an investigation on feature selection techniques', *Softw., Pract. Exp.*, 2011, **41**, (5), pp. 579–606
[43] Yu, Q., Jiang, S., Zhang, Y.: 'The performance stability of defect prediction models with class imbalance: an empirical study', *IEICE Trans. Inf. Syst.*, 2017, **100-D**, (2), pp. 265–272
[44] Khoshgoftaar, T.M., Gao, K., Napolitano, A., *et al.*: 'A comparative study of iterative and non-iterative feature selection techniques for software defect prediction', *Inf. Syst. Front.*, 2014, **16**, (5), pp. 801–822
[45] Sun, Z., Song, Q., Zhu, X., *et al.*: 'A novel ensemble method for classifying imbalanced data', *Pattern Recogn.*, 2015, **48**, (5), pp. 1623–1637
[46] Huang, J., Ling, C.X.: 'Using AUC and accuracy in evaluating learning algorithms', *IEEE Trans. Knowl. Data Eng.*, 2005, **17**, (3), pp. 299–310
[47] Jiang, Y., Lin, J., Cukic, B., *et al.*: 'Variance analysis in software fault prediction models'. Proc. Int. Symp. Software Reliability Engineering, Mysuru, Karnataka, India, November 2009, pp. 99–108
[48] Catal, C., Diri, B.: 'Investigating the effect of dataset size, metrics sets, and feature selection techniques on software fault prediction problem', *Inf. Sci.*, 2009, **179**, (8), pp. 1040–1058
[49] Czibula, G., Marian, Z., Czibula, I.G.: 'Software defect prediction using relational association rule mining', *Inf. Sci.*, 2014, **264**, pp. 260–278
[50] Zhou, Y., Xu, B., Leung, H., *et al.*: 'An in-depth study of the potentially confounding effect of class size in fault prediction', *ACM Trans. Softw. Eng. Method.*, 2014, **23**, (1), pp. 10:1–10:51
[51] Laradji, I.H., Alshayeb, M., Ghouti, L.: 'Software defect prediction using ensemble learning on selected features', *Inf. Softw. Technol.*, 2015, **58**, pp. 388–402
[52] Nam, J., Kim, S.: 'CLAMI: defect prediction on unlabeled datasets'. Proc. Int. Conf. Automated Software Engineering, Lincoln, NE, USA, November 2015, pp. 452–463
[53] Nam, J., Kim, S.: 'Heterogeneous defect prediction'. Proc. Joint Meeting on Foundations of Software Engineering, Bergamo, Italy, 30 August – 4 September 2015, pp. 508–519
[54] Malhotra, R., Khanna, M.: 'An empirical study for software change prediction using imbalanced data', *Empir. Softw. Eng.*, 2017, **22**, (6), pp. 2806–2851
[55] Wilcoxon, F.: 'Individual comparisons by ranking methods', *Biom. Bull.*, 1945, **1**, (6), pp. 80–83
[56] Demšar, J.: 'Statistical comparisons of classifiers over multiple data sets', *J. Mach. Learn. Res.*, 2006, **7**, pp. 1–30
[57] Cohen, J.: '*Statistical power analysis for the behavioral sciences*' (Lawrence Erlbaum Associates, Hillsdale, NJ 1988, 2nd edn.)